



05. 복사, 이동과 값 의미

Heesung Oh

<https://github.com/3dapi>



- 값 의미와 객체 정체성
- 기본 복사와 복사 정책
- 얇은 복사, 깊은 복사, 이중 해제
- 복사 생성자와 복사 대입 연산자
- Rule of Three
- 값 범주와 우측값 참조
- 이동 생성자와 이동 대입 연산자
- `std::move`, `noexcept`
- Rule of Five, Rule of Zero
- 복사 생략과 값 반환

객체 복사



값을 복제할 것인가?

자원을 공유할 것인가?

소유권을 이전할 것인가?

복사를 금지할 것인가?



- 값은 계산과 비교의 대상이 되는 내용
- 객체는 값을 저장하는 메모리 영역과 수명을 가진 실체
- 값이 같다는 사실과 같은 객체라는 사실은 다름

```
int first = 10;  
int second = 10;  
  
std::cout << std::boolalpha;  
std::cout << (first == second) << '\n';  
std::cout << (&first == &second) << '\n';  
// 출력:  
// true  
// false
```

비교	의미
<code>first == second</code>	저장된 값 비교
<code>&first == &second</code>	같은 객체인지 주소 비교

- 두 객체가 같은 값을 저장해도 서로 다른 객체일 수 있음



● 값 의미

```
Point original{10, 20};  
Point copy = original;  
  
copy.x = 100;
```

- **copy**를 변경해도 **original**은 유지
- 복사본은 원본과 독립된 객체

● 참조 의미

```
Point point{10, 20};  
  
Point* first = &point;  
Point* second = first;  
  
second->x = 100;
```

- **first**와 **second**는 같은 객체를 가리킴
- 한쪽을 통해 변경하면 같은 대상에 반영



5.1.4 값 의미를 가지는 타입

- 복사본이 원본과 독립적으로 동작
- 객체의 값에 따라 동일성 판단 가능
- 함수 인수와 반환값으로 자연스럽게 사용 가능
- 객체가 자신의 자원을 스스로 관리
- 객체 수명이 끝나면 소유 자원도 정리

```
std::string first = "Player";  
std::string second = first;
```

```
second.append("_Two");
```

first → "Player"

second → "Player_Two"

타입 예	값 의미
좌표	같은 좌표값이면 같은 값
날짜	같은 연월일이면 같은 값
문자열	같은 문자 내용이면 같은 값

- std::string은 내부적으로 동적 메모리를 사용
- 복사본 변경이 원본 문자열에 영향을 주지 않음



5.1.5 자원을 소유하는 객체

- 객체가 자원의 사용과 해제를 책임지면 자원을 소유한다고 표현
- 동적 메모리, 파일, 소켓, 운영체제 핸들 등이 대표적
- 포인터 값만 복사하면 자원 소유 규칙이 깨질 수 있음

```
class IntArray
{
public:
    explicit IntArray(std::size_t size)
        : size(size),
          data(size > 0 ? new int[size]{} : nullptr)
    {}
    ~IntArray()
    {
        delete[] data;
    }
private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

IntArray 객체

├ size
└ data ──▶ 동적 배열

- data에는 주소값만 저장
- 클래스 의미에서는 동적 배열까지 객체가 관리해야 할 상태



- 명시적 복사 선언 없으면 컴파일러가 멤버 단위 복사 제공
- 기본형과 값 의미 멤버에는 자연스러운 동작
- 자원을 가리키는 포인터 멤버에는 문제가 생길 수 있음

```
struct Point
{
    int x, y;
};

Point first{10, 20};
Point second = first;

second.x = first.x
second.y = first.y
```

- `name`에는 `std::string`의 복사 동작 적용
- `position`에는 `Point`의 복사 동작 적용



- 포인터가 가리키는 자원의 내용은 복제 안함.
주소값만 복사
- 두 객체가 같은 자원을 가리키는 상태
- 객체가 자원을 소유한다면 위험

```
class IntArray
{
    // ...
private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

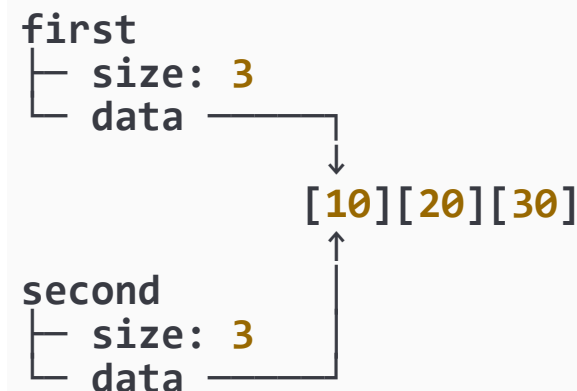
```
IntArray first(3);
```

```
first.Set(0, 10);
first.Set(1, 20);
first.Set(2, 30);
```

```
IntArray second = first;
```

```
second.Set(0, 100);
std::cout << first.Get(0) << '\n';
```

- 출력값은 100
- 복사본 변경이 원본에도 나타남





- 원본 객체가 소유한 자원의 내용을 새로운 자원에 복제
- 두 객체는 같은 값을 가지지만 서로 다른 자원을 소유
- 한 객체의 변경과 소멸이 다른 객체에 영향을 주지 않음

```
first
├ size: 3
└ data → [10][20][30]
```

```
second
├ size: 3
└ data → [10][20][30]
```

깊은 복사 작업

- 원본과 같은 크기의 자원 준비
- 원본 자원의 내용을 새 자원으로 복사
- 복사본이 새 자원을 소유하도록 설정
- 각 객체가 자신의 자원만 해제하도록 유지
- 복사 생성자와 복사 대입 연산자에서 구현

복사 방식	결과
얕은 복사	주소값 복사, 자원 공유 가능
깊은 복사	새 자원 준비, 내용 복제



복사 정책	복사 결과	대표 용도
깊은 복사	독립된 자원과 같은 내용	배열, 문자열, 이미지 버퍼
공유 소유	하나의 자원을 함께 소유	공유 데이터, 캐시된 리소스
비소유 참조	같은 대상을 참조	뷰, 관찰자, 임시 접근 객체
복사 금지	복사 시 컴파일 오류	고유 핸들, 잠금, 단일 연결

```
class PlayerView
{
public:
    explicit PlayerView(Player* player)
        : player(player) {}
private:
    Player* player = nullptr; // 소유하지 않음
};
```

```
class UniqueRsc
{
public:
    UniqueRsc(const UniqueRsc&) = delete;
    UniqueRsc& operator=(const UniqueRsc&) = delete;
};
```

- 포인터 멤버는 항상 깊은 복사를 해야 하는 것은 아님
- 포인터의 역할과 클래스의 의미에 따라 정책 결정



5.2.5 자명한 복사와 바이트 단위 복사

- C++ 객체 복사에 무조건 `memcpy()`를 사용하면 안 됨
- 자명하게 복사 가능한 타입은 객체 표현을 바이트 단위로 복사 가능
- C++ 객체는 각 멤버의 복사 생성자, 복사 대입 연산자 사용

```
Point first{10, 20};  
Point second{};  
std::memcpy(&second, &first, sizeof(Point));  
#include <type_traits>  
static_assert(std::is_trivially_copyable_v<Point>);  
struct Player  
{  
    std::string name;  
    int hp;  
};  
static_assert(!std::is_trivially_copyable_v<Player>);
```

- `std::string`은 내부 자원을 관리
- 바이트 단위 복사는 자원 관리 규칙을 깨뜨릴 수 있음
- 패딩 바이트 때문에 바이트 표현 비교가 값 비교와 같다고 볼 수도 없음



- 얇은 복사로 두 객체가 같은 동적 배열을 소유한다고 판단하면 소멸 과정에서 문제 발생
- 같은 동적 메모리를 두 번 해제하는 동작은 이중 해제
- 결과는 미정의 동작

```
{  
    IntArray first(3);  
    IntArray second = first;  
}
```

second 소멸 → 배열 해제
first 소멸 → 이미 해제된 배열 다시 해제

잘못된 복사와 소유권 처리의 결과

- 이중 해제
- 댕글링 포인터
- 메모리 누수
- 비소유 포인터의 잘못된 해제
- new[]와 delete의 잘못된 조합
- 단일 객체와 delete[]의 잘못된 조합
- 먼저 클래스가 자원을 소유하는지 판단
- 복사 정책에 맞는 특별 멤버 함수 구현



- 같은 타입의 다른 객체를 사용하여 새 객체를 초기화
- 일반적으로 같은 클래스 타입의 상수 참조를 매개변수로 받음
- 생성자이므로 반환 타입 없음

```
class IntArray
{
public:
    IntArray(const IntArray& other);
};
```

```
IntArray first(3);
IntArray second(first);
IntArray third = first;
```

구성	의미
IntArray	생성자 이름
const IntArray&	원본 객체를 읽기 전용 참조로 전달
other	원본 객체를 나타내는 매개변수

- 상수 참조를 사용하면 원본을 다시 복사하지 않음
- 복사 과정에서 원본을 변경하지 않는다는 의도 표현



- 자원을 직접 소유하는 객체는 복사 생성자에서 깊은 복사 구현
- 원본의 크기 복사
- 같은 크기의 새 배열 할당
- 원본 배열의 원소를 새 배열로 복사

```
IntArray::IntArray(const IntArray& other)
: size(other.size),
  data(other.size > 0 ? new int[other.size] : nullptr)
{
    for (std::size_t i = 0; i < size; ++i)
    {
        data[i] = other.data[i];
    }
}
```

```
IntArray second = first;
second.Set(0, 100);

std::cout << first.Get(0) << '\n';
std::cout << second.Get(0) << '\n';
```

원본 크기 확인



새 배열 할당



원소 복사



복사본이 새 배열 소유



- 복사 생성자의 매개변수를 값으로 받을 수 없음
- 값 매개변수 자체를 만들기 위해 다시 복사 생성자가 필요

```
// 올바른 복사 생성자가 될 수 없음  
IntArray(IntArray other);
```

- 일반적인 복사 생성자는 const 상수 참조 사용

```
IntArray(const IntArray& other);
```

```
IntArray normal(3);  
const IntArray fixed(3);  
IntArray first(normal);  
IntArray second(fixed);
```

```
// 특별한 이유가 없다면 사용하지 않음  
IntArray(IntArray& other);
```



- 대입 대상 객체가 이미 자원을 소유하고 있을 수 있음
- 기존 주소를 잃으면 메모리 누수 발생
- 기존 자원을 너무 먼저 해제하면 예외와 자기 대입에서 문제 발생

```
IntArray first(3);  
IntArray second(100);  
  
second = first;
```

```
// 잘못된 구현  
size = other.size;  
data = new int[other.size];
```

```
// 아직 안전하지 않은 구현  
delete[] data;  
  
data = new int[other.size];  
size = other.size;
```

- 이전 data 주소를 잃어버림
- 기존 배열에 다시 접근할 방법 없음

- 새 배열 할당 실패 시 기존 값 손실
- 자기 대입이면 원본 데이터도 먼저 해제됨



- 객체가 자기 자신에게 대입될 수 있음
- this와 &other가 같은 주소이면 같은 객체
- 기존 자원을 먼저 해제하는 구현은 원본 데이터까지 해제

```
array = array;
```

```
IntArray& IntArray::operator=(const IntArray& other)
{
    delete[] data;
```

- 자기 대입 검사 필요
- 새 자원을 안전하게 준비한 뒤 기존 자원을 교체하도록 구성

```
IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other)
    {
        return *this;
    }
```

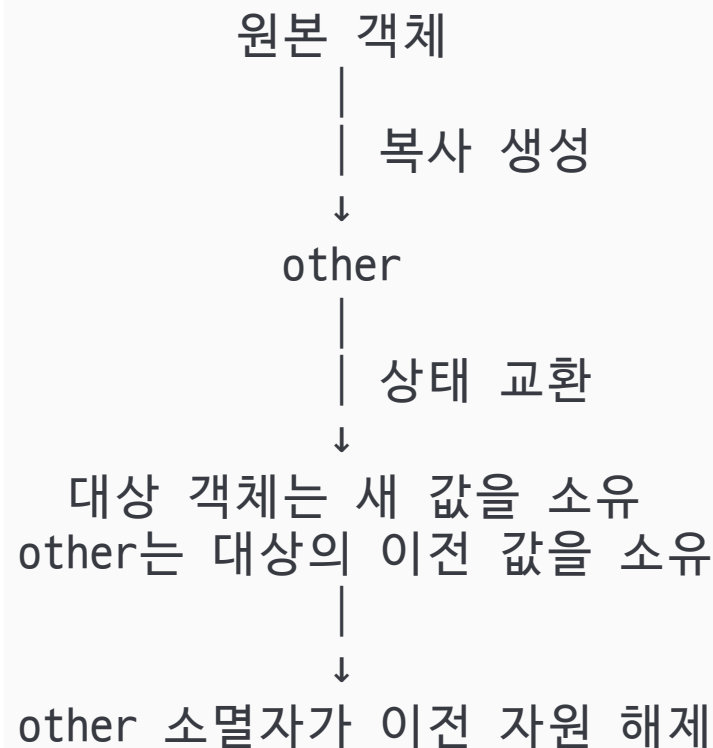




- 원본의 복사본을 먼저 만든 뒤 대상 객체의 상태와 교환
- 복사본 생성에 실패하면 대입 대상은 변경되지 않음
- 자기 대입에서도 별도 검사 없이 올바른 결과 가능

```
#include <utility>
class IntArray
{
public:
    IntArray& operator=(IntArray other)
    {
        Swap(other);
        return *this;
    }
    void Swap(IntArray& other) noexcept
    {
        std::swap(size, other.size);
        std::swap(data, other.data);
    }

private:
    std::size_t size = 0;
    int* data = nullptr;
};
```





- 소멸자, 복사 생성자, 복사 대입 연산자 중 하나를 직접 작성해야 한다면 나머지도 함께 검토
- 세 함수는 하나의 자원 소유 정책을 서로 다른 시점에 적용
- 무조건 세 함수를 모두 작성하라는 뜻은 아님
- 컴파일러 기본 동작으로 소유 규칙이 유지되는지 확인하는 원칙

```
~IntArray();  
IntArray(const IntArray& other);  
IntArray& operator=(const IntArray& other);
```

특별 멤버 함수	호출 상황	자원 관리 역할
소멸자	객체 수명 종료	소유한 자원 해제
복사 생성자	새 객체를 기존 객체로 초기화	독립된 자원 준비와 내용 복사
복사 대입 연산자	기존 객체의 값 교체	기존 자원 정리와 새 내용 복사

소멸자 → 현재 소유 자원 해제
복사 생성자 → 새 객체의 자원 준비
복사 대입 → 기존 객체의 자원 교체



- 생성 과정에서 확보한 자원을 정확히 해제
- 복사본이 독립된 자원을 소유하도록 구성
- 대입 대상의 자원은 유지
- 자기 대입에서도 올바르게 동작
- 자원 준비 실패해도 기존 객체 사용
- 이동을 지원하려면 소유권 이전 규칙도 추가
- 특별 멤버 함수의 추가와 수정이 서로 일관

직접 자원 관리
↓
소멸자
복사 생성자
복사 대입 연산자
이동 생성자
이동 대입 연산자
예외 안전성

- 가능하면 `std::string`, `std::vector`처럼
자원 관리를 이미 구현한 타입을 멤버로 사용



5.6.2 좌측값 (lvalue)

- 객체나 함수의 정체성을 나타내는 표현식
- 이름을 가진 변수 표현식은 일반적으로 lvalue
- 참조를 반환하는 함수 호출도 lvalue가 될 수 있음
- 수정 가능 여부와 값 범주는 별개의 성질

```
int value = 10;  
value = 20;  
std::cout << value << '\n';
```

```
int& Select(int& value)  
{  
    return value;  
}
```

```
int number = 10;  
Select(number) = 100;  
const int maxCount = 100;
```

표현식	값 범주	수정 가능 여부
value	lvalue	가능
Select(number)	lvalue	가능
maxCount	lvalue	불가능



5.6.3 prvalue (pure rvalue: 순수우측값)

- 값을 계산하거나 결과 객체를 초기화하는 표현식
- 리터럴, 산술 결과, 값을 반환하는 함수 호출이 대표적
- 클래스 타입 prvalue는 결과 객체를 직접 초기화할 수 있음

```
#include <iostream>
#include <type_traits>

int main()
{
    std::cout << std::boolalpha;

    int first = 100, second = 200;
    42;
    3.14;
    first + second;

    std::cout << "prvalue Example\n";
    std::cout << "42 : " << std::is_same_v<decltype((42)), int> << '\n';
    std::cout << "3.14 : " << std::is_same_v<decltype((3.14)), double> << '\n';
    std::cout << "first + second : " << std::is_same_v<decltype((first + second)), int> << '\n';
}
```



- move: 자원의 소유권 이전
- Identity: 객체 식별
- xvalue: Identity 이 있고, move 가능한 표현식
- rvalue: move 가능한 표현식

```
IntArray source(100);  
IntArray target(std::move(source));
```

```
source  
├ 객체의 정체성 있음  
└ 표현식은 lvalue  
  
std::move(source)  
├ 같은 객체의 정체성 있음  
└ 이동 대상으로 사용할 수 있는 xvalue  
  
10 + 20 // rvalue - prvalue  
std::move(source) // rvalue - xvalue
```





- rvalue 참조는 &&로 선언
- rvalue 에 결합
- 이동 생성자와 이동 대입 연산자의 핵심 매개변수
- rvalue 참조는 복사나 move 할 수 있는 객체로 구분 가능

```
int&& value = 10;
```

```
IntArray(IntArray&& other) noexcept;
```

```
void Process(const IntArray& array); // 읽기 또는 복사
```

```
void Process(IntArray&& array); // 이동 가능한 객체
```

매개변수	받는 표현식 종류	의미
const T&	lvalue, rvalue	읽기 또는 복사
T&&	rvalue	이동 가능
T&	수정 가능한 lvalue	원본 변경 가능



- 우측값 참조 타입으로 선언한 변수라도 이름을 사용한 표현식은 lvalue
- 같은 객체에 반복해서 접근할 수 있기 때문
- 이동 생성자 내부에서도 같은 규칙 적용

```
IntArray&& reference = IntArray(10);  
// lvalue, 복사  
IntArray copied(reference);  
  
// xvalue로 변환, 이동  
IntArray moved(std::move(reference));  
  
IntArray::IntArray(IntArray&& other)  
{  
    // 표현식 other: lvalue  
}
```

```
class Player  
{  
public:  
    Player(Player&& other) noexcept  
        : name(std::move(other.name)),  
          items(std::move(other.items)) {}  
private:  
    std::string name;  
    std::vector<int> items;  
};
```

- other.name과 other.items도 이름을 통해 접근하므로 lvalue
- 멤버를 이동하려면 다시 std::move() 필요



- 계산 결과, 함수 반환에서 이름 없는 임시 객체 생성 가능
- 상수 lvalue 참조는 임시 객체에 결합하여 수명 연장 가능
- rvalue 참조도 직접 초기화한 임시 객체의 수명을 연장 가능
- 지역 객체의 참조를 반환한다고 수명이 연장되지 않음

```
IntArray array = IntArray(10);  
const IntArray& reference = IntArray(10);  
IntArray&& reference = IntArray(10);  
const IntArray& CreateInvalidReference()  
{  
    IntArray local(10);  
    return local; // 잘못된 참조 반환  
}
```

- 함수가 끝나면 local은 소멸
- 반환된 참조는 더 이상 유효하지 않음





- 같은 타입의 우측값 참조를 받아 새 객체를 초기화
- 원본 객체의 자원 주소와 크기를 가져옴
- 원본 객체를 자원을 소유하지 않는 상태로 변경
- 보통 예외를 던지지 않으면 `noexcept`

```
IntArray::IntArray(IntArray&& other)
```

noexcept

```
    : size(other.size),  
      data(other.data)
```

```
{
```

```
    other.size = 0;
```

```
    other.data = nullptr;
```

```
}
```

```
IntArray source(1000);
```

```
IntArray target(std::move(source));
```

target

└ size: 1000

└ data —▶ [동적 배열]

source

└ size: 0

└ data: nullptr

- 배열 원소 1000개를 복사하지 않음
- size와 data의 소유권만 이전



- 이미 존재하는 객체의 기존 자원을 정리
- 다른 객체의 자원을 넘겨받음
- 원본 객체를 유효한 빈 상태로 변경
- 현재 객체의 참조 반환

```
IntArray& IntArray::operator=(IntArray&& other) noexcept
{
    if (this == &other)
    {
        return *this;
    }
    delete[] data;
    size = other.size;
    data = other.data;
    other.size = 0;
    other.data = nullptr;
    return *this;
}

IntArray first(100);
IntArray second(1000);
first = std::move(second);
```

first의 기존 배열 해제
↓
second의 size, data
first로 이전
↓
second를 빈 상태로 변경



- 이동된 원본 객체는 수명이 끝난 객체가 아님
- 정상적으로 소멸할 수 있어야 함
- 새로운 값을 대입하여 다시 사용할 수 있어야 함
- 타입이 이동 후 상태를 명시할 수 있음

이동 후 객체	의미
수명	계속 존재
소멸	정상적으로 가능해야 함
재사용	새 값 대입 가능해야 함
값	타입의 보장에 따라 다름





- 인수가 lvalue이면 복사 생성자 선택
- 인수가 rvalue이면 이동 생성자 선택 가능
- 대입에서도 같은 원리 적용

```
IntArray source(10);  
// 복사  
IntArray first(source);  
// 이동 or 복사  
IntArray second(IntArray(10));  
// 이동  
IntArray third(std::move(source));  
IntArray target(3);  
IntArray source(10);
```

```
// 복사 대입  
target = source;  
// 이동 대입  
target = std::move(source);
```

구분	복사	이동
원본 표현식	주로 lvalue	rvalue 또는 std::move() 결과
자원 처리	자원 생성, 내용 복제	기존 자원의 소유권 이전
원본 상태	유지	유효하지만 값이 변경될 수 있음
비용	자원 크기에 비례 가능	포인터와 크기 이전 정도 가능
대표 함수	복사 생성자, 복사 대입	이동 생성자, 이동 대입



- `std::move()`는 객체의 `const`를 제거하지 않음
- 일반적인 이동 생성자는 비 상수 우측값 참조를 받음
- 이동은 원본 객체를 빈 상태로 바꾸는 등 원본 변경이 필요할 수 있음
- `const` 객체는 보통 이동되지 않고 복사될 수 있음

```
const IntArray source(10);
```

```
IntArray target(std::move(source));
```

```
IntArray(IntArray&& other) noexcept;
```

```
IntArray(const IntArray& other);
```

```
std::move(nonConstObject) → T&& → 이동 가능
```

```
std::move(constObject) → const T&& → 일반적으로 복사
```

- `const T&&`를 받는 별도 함수를 만들 수는 있음
- 원본을 변경할 수 없어 자원 소유권 이전 불가
- 이동할 객체에는 불필요한 `const`를 붙이지 않음



5.8.2 std::move가 이동을 보장하지 않는 이유

- std::move()를 호출해도 이동 생성자나 이동 대입 연산자가 없으면 복사가 선택될 수 있음
- 실제 호출은 오버로드 해석으로 결정
- std::move()는 이동 함수가 선택될 기회를 제공

```
class CopyOnly
{
public:
    CopyOnly() = default;
    CopyOnly(const CopyOnly&) = default;
};
```

```
CopyOnly first;
CopyOnly second(std::move(first));
```

- 이동 생성자 없음
- const CopyOnly&, rvalue에 결합 가능
- 복사 생성자 사용 가능

```
void Process(const IntArray& array);
void Process(IntArray&& array);
```

```
IntArray values(10);
```

```
Process(values);           // const IntArray& 버전
Process(std::move(values)); // IntArray&& 버전
```




5.8.3 noexcept 이동

- 이동 생성자와 이동 대입 연산자가 예외를 밖으로 전달하지 않으면 noexcept 지정 가능
- noexcept는 함수 계약의 일부
- noexcept 함수 밖으로 예외가 빠져나가면 std::terminate() 호출
- 실제 예외 가능성이 있는 이동 함수에 무조건 붙이면 안 됨

```
IntArray(IntArray&& other) noexcept;  
IntArray& operator=(IntArray&& other) noexcept;
```

```
IntArray::IntArray(IntArray&& other) noexcept  
: size(other.size), data(other.data)  
{  
    other.size = 0; other.data = nullptr;  
}
```

```
class Wrapper  
{  
public:  
    Wrapper(Wrapper&&) noexcept(  
        std::is_nothrow_move_constructible_v<std::string>) = default;  
private:  
    std::string value;  
};
```

- 이동 중 메모리 할당이나 실패 가능한 작업이 없다면 noexcept 우선 검토



5.8.4 move_if_noexcept와 컨테이너 재배포

- `std::move_if_noexcept()`
이동이 안전하면 이동 가능한 참조 제공
- 이동이 예외를 던질 수 있고 복사가 가능하면
복사가 선택되도록 상수 참조 제공
- 표준 컨테이너의 원소 재배포 원리 이해에 중요

```
T target(std::move_if_noexcept(source));
```

이동 생성자가 **noexcept**인가?

- └ 예 → 이동 가능
- └ 아니오
 - └ 복사 가능 → 복사 가능하도록 **const** 참조 제공
 - └ 복사 불가 → 이동 사용

std::vector 재배포

기존 저장 공간 [A][B][C]



더 큰 저장 공간 [A][B][C][][][]



5.8.6 반환값에 std::move를 사용할 때의 문제

- NRVO: Named Return Value Optimization(반환 값 최적화)
- 지역 객체를 값으로 반환할 때 일반적으로 std::move()를 붙이지 않음
- 컴파일러는 NRVO를 적용해 결과 위치에 직접 생성할 수 있음
- NRVO가 적용되지 않아도 지역 객체는 이동 대상으로 처리될 수 있음
- 불필요한 std::move()는 NRVO를 방해할 수 있음

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);
    return result;
}
```

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);
    return std::move(result); // 비 권장
}
```

return result;



NRVO 가능



NRVO 미적용 시 이동 가능

return std::move(result);



NRVO 대상에서 제외될 수 있음



5.9 특별 멤버 함수의 설계

- 객체의 생성, 복사, 이동, 소멸을 담당하는 특별 멤버 함수
- 컴파일러 기본 동작을 사용할지, 직접 구현할지, 호출을 금지할지 결정

특별 멤버 함수	암시적 선언 조건	암시적 삭제 대표 원인
기본 생성자	사용자 선언 생성자 없음	기본 생성 불가 멤버·기반 ex) 초기값 없는 <code>const</code> ·참조 멤버
소멸자	사용자 선언 소멸자 없음	멤버·기반 소멸자 사용 불가 ex) 멤버·기반 소멸자 = <code>delete</code>
복사 생성자	사용자 선언 복사 생성자 없음	멤버·기반 복사 생성 불가 ex) 멤버·기반 복사 생성자 = <code>delete</code>
복사 대입	사용자 선언 복사 대입 없음	<code>const</code> ·참조 멤버, 멤버·기반 복사 대입 불가
이동 생성자	복사 생성·복사 대입· 이동 대입·소멸자 없음	멤버·기반 이동 생성 불가
이동 대입	복사 생성·복사 대입· 이동 생성·소멸자 없음	<code>const</code> ·참조 멤버, 멤버·기반 이동 대입 불가



5.9.1 암시적 이동 선언에 영향을 주는 함수

- 이동 생성자·이동 대입: 특정 특별 멤버 함수 미선언 시 암시적 선언
- 사용자 선언 소멸자 존재: 이동 함수의 암시적 선언 억제
- 빈 소멸자도 사용자 선언 소멸자에 해당

```
class Data
{
public:
    ~Data(){}
private:
    std::string value;
};
```

```
class Data
{
public:
    Data() = default;
    ~Data() = default;
    Data(const Data&) = default;
    Data& operator=(const Data&) = default;
    Data(Data&&) noexcept = default;
    Data& operator=(Data&&) noexcept = default;
private:
    std::string value;
```

- 기본 소멸 동작 충분 → 소멸자 선언 생략
- 복사·이동 정책 중요 → 관련 특별 멤버 함수 명시



5.9.3 Rule of Zero

- Zero → 5대 특별 멤버 함수 직접 구현 0개
 - ◆ 자원 관리 타입 멤버 사용
 - ◆ 특별 멤버 함수 직접 작성 최소
 - ◆ 상태와 기능 설계에 집중
 - ◆ `std::string`, `std::vector` 등 활용
 - ◆ 복사·이동·자원 해제 → 멤버 타입이 담당

```
class Inventory
{
public:
    void AddItem(const std::string& item)
    {
        items.push_back(item);
    }

```

```
private:
    std::string owner;
    std::vector<std::string> items;
};
```

```
Inventory first;
Inventory second = first;
Inventory third = std::move(first);
```

직접 자원 관리	Rule of Zero
new, delete 직접 관리	자원 관리 타입 활용
특별 멤버 함수 직접 구현	특별 멤버 함수 생략 가능
소유권·수명 직접 관리	멤버 타입이 소유권·수명 관리



- 복사가 의미 없는 타입
- 복사 생성자 삭제
- 복사 대입 연산자 삭제
- 고유 자원 중복 소유 방지
- 싱글톤 → 생성 제한 + 복사 금지 가능

```
class UniqueFile
{
public:
    UniqueFile() = default;

    UniqueFile(const UniqueFile&) = delete;
    UniqueFile& operator=(const UniqueFile&) =
delete;
};

UniqueFile first;
UniqueFile second;

// UniqueFile third = first; // 오류
// second = first;           // 오류
```

타입 성격	복사 정책
고유 핸들	복사 금지
잠금 객체	복사 금지
단일 연결	복사 금지
복제 의미가 불명확한 자원	복사 금지





- 복사 금지 → 이동 허용
- 단일 소유자 유지
- 소유권 이전 허용
- 함수 반환 가능
- 컨테이너 저장 가능
- 대표 타입 → `std::unique_ptr`

```
class UBuffer
{
public:
    explicit UBuffer(std::size_t size)
        : size(size),
          data(size > 0 ? new int[size]{} : nullptr)
    {
    }
    ~UBuffer()
    {
        delete[] data;
    }
    UBuffer(const UBuffer&) = delete;
    UBuffer& operator=(const UBuffer&) = delete;
};
```

```
UBuffer(UBuffer&& other) noexcept
    : size(other.size),
      data(other.data) {
    other.size = 0;
    other.data = nullptr;
}

private:
    std::size_t size = 0; int* data = nullptr;
};

UBuffer CreateBuffer()
{
    return UBuffer(1024);
}
```





5.9.6 = default와 = delete의 적용

- = default → 컴파일러 기본 동작 사용
- = delete → 특정 함수 사용 금지
- 클래스 의미와 기본 동작 일치 여부 확인
- 원시 포인터 소유 클래스 → 기본 복사 주의
- 원시 포인터 기본 복사 → 얇은 복사 가능

```
struct Point
{
    int x = 0, y = 0;

    Point() = default;
    Point(const Point&) = default;
    Point& operator=(const Point&) = default;
};
```

```
class Rsc
{
public:
    Rsc() = default;

    Rsc(const Rsc&) = delete;
    Rsc& operator=(const Rsc&) = delete;

    Rsc(Rsc&&) noexcept = default;
    Rsc& operator=(Rsc&&) noexcept = default;
};
```

문법	의미
= default	컴파일러 기본 동작 요청
= delete	해당 함수 호출 금지



- 객체 초기화·반환 과정의 복사/이동 생략
- 불필요한 중간 객체 생성 감소
- 반환 객체 직접 생성 가능
- 새 객체 생성 함수 → 값 반환 우선 검토
- 지역 객체 포인터·참조 반환 → 수명 오류 위험
- 값 반환
 - ◆ 반환 객체 → 호출자의 결과 객체 위치에 직접 생성 가능
 - ◆ 불필요한 복사·이동 제거 가능
 - ◆ 반환값의 소유권 명확
 - ◆ 지역 객체 수명 문제 방지

```
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}
```

```
IntArray values = CreateArray(1000);
```

CreateArray()의 반환 결과
↓ 직접 초기화
values



• Named Return Value Optimization

- ◆ 이름 있는 지역 객체 반환 시 적용 가능
- ◆ 반환 객체 위치에 지역 객체 직접 생성
- ◆ 복사·이동 생략 가능
- ◆ 적용 보장 아님
- ◆ 여러 지역 객체 반환 → NRVO 적용 어려움
- ◆ NRVO 미적용 → 이동 가능

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);
    return result;
}
```

```
// NRVO 적용
// result가 결과 위치에 직접 생성

IntArray SelectArray(bool selectFirst)
{
    IntArray first(10);
    IntArray second(20);
    if (selectFirst)
    {
        return first;
    }
    return second;
}
```





- 이름 있는 지역 객체 반환
- NRVO 적용 가능
- NRVO 미적용 → 암시적 이동 가능
- `return result;` 사용 권장
- `std::move(result)` 직접 사용 불필요
- `std::move(result)` → NRVO 방해 가능
- 이동 불가 → 조건에 따라 복사 시도
- 매개변수·멤버·전역 객체 반환 → 별도 규칙 적용

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);
    return result;
}
```

NRVO 적용

└─ `result`가 결과 위치에 직접 생성

NRVO 미적용

└─ 반환 대상 지역 객체를 이동 가능한 대상으로 처리
└─ 이동이 불가능하면 조건에 따라 복사 시도

// 권장

`return result;`

// NRVO를 방해할 수 있음

`return std::move(result);`



- 큰 객체 값 반환 ≠ 항상 고비용
- 복사 생략 가능
- 반환 객체 직접 생성 가능
- 복사 생략 불가 → 이동 활용 가능
- 값 반환 → 소유권 명확
- 값 반환 → 수명 관리 단순화
- 지역 객체 포인터·참조 반환 오류 방지

```
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}
```

```
IntArray CreateArray(std::size_t size);
```

상황	일반적인 선택
작은 값 전달	값 전달
큰 객체 읽기	const 참조 전달
새 값 생성·반환	값 반환
기존 객체 읽기 전용 노출	수명이 보장되는 const 참조 반환
소유권 이전	우측값 참조 + 이동



- 표현식의 값 범주는 객체의 정체성과 이동 가능성을 나타냄
- 이름 있는 객체 표현식은 일반적으로 lvalue
- 계산 결과는 prvalue
- `std::move()`는 객체를 이동 가능한 xvalue로 취급하게 함
- 우측값 참조 타입의 변수도 이름을 사용한 표현식은 lvalue
- 이동 생성자와 이동 대입 연산자는 자원의 내용을 복제하지 않고 소유권 이전
- 이동된 원본 객체는 여전히 유효하며 소멸과 새 값 대입이 가능해야 함
- `std::move()`는 실제 이동을 수행하지 않고 값 범주를 변환
- 이동 함수가 예외를 발생시키지 않으면 `noexcept` 명시
- C++11 이후 Rule of Five로 복사와 이동을 함께 검토
- 자원 관리 타입을 멤버로 사용하면 Rule of Zero 적용 가능
- 지역 객체 반환에는 불필요한 `std::move()`를 사용하지 않음





glvalue, rvalue, lvalue, xvalue, prvalue

값범주		의미	객체 식별	move 가능	decltype ((expr))
glvalue - 객체 식별(identity) 기준 - 객체 식별 가능한 표현식 - lvalue $\cup$ xvalue	lvalue	glvalue 중에서 move 불가	O	X	T&
	xvalue glvalue $\cap$ rvalue	glvalue, rvalue 중에서 객체 식별 가능하고, move 가능한 표현식	O	O	T&&
rvalue - move 기준 - move 가능한 표현식 - prvalue $\cup$ xvalue	prvalue	rvalue 중에서 값을 생성하는 표현식	X	O	T

값범주	샘플코드	
lvalue	value; GetReference(); reference; rvalueReference;	// lvalue : 이름 있는 객체의 표현식 // lvalue : int& 반환 함수 호출 결과 // lvalue 참조 변수 // 이름이 있는 rvalue 참조 변수
xvalue	std::move(value); GetRvalueReference();	// xvalue : 객체 식별과 move 가능 // xvalue : int&& 반환 함수 호출 결과
prvalue	100 ; value + 10; GetValue();	// 리터럴 // 연산 결과 // int 반환